



## The Team:

1. Theart Jooste [601288] – Project Leader
2. Jason Crous [601662] – Project Manager
3. Rivan Martiz [601530] – AI & ML Engineering Team Lead
4. Nicholas Vronka [601596] – AI & ML Engineer
5. Jonathan Rossouw [601761] – Front End Development Team Lead
6. Ryno Lourens [601845] - Front End Developer
7. Marcus Swanepoel [602162] – Backend Developer Team Lead
8. Declin Vorkel [601756] - Backend Developer
9. Timothy Wubbeling [602543] – Data Engineer Team Lead
10. Rudi Jan Du Plessis [601342] – Data Engineer Team

## Table of Contents

|                                          |    |
|------------------------------------------|----|
| Project Background and Motivation .....  | 3  |
| Project Literature Review .....          | 4  |
| Non-Functional Requirements .....        | 5  |
| Functional Requirements .....            | 8  |
| Technical Constraints .....              | 9  |
| Data Requirements: .....                 | 13 |
| User Interface Skeleton .....            | 14 |
| Data Architecture Specification .....    | 16 |
| Testing & Validation Criteria .....      | 19 |
| System Architecture and Design .....     | 23 |
| Hardware List .....                      | 33 |
| Map Each Role's Journey .....            | 34 |
| Identify Key Input & Output Points ..... | 36 |
| Project Timeline .....                   | 37 |
| References .....                         | 38 |

## Project Background and Motivation

South Africa has one of the world's highest drowning rates, especially during peak holiday seasons when coastal locations are congested. Traditional lifeguard monitoring is limited by poor visibility, slow response times, and insufficient staff to cover long areas of shoreline. These difficulties are increased by unpredictable ocean currents, rip tides, and abrupt weather changes, making rescues both time-sensitive and complex.

Recent accidents along the KZN South Coast and Hermanus illustrate the critical need for creative solutions to improve situational awareness and decrease mortality. While lifeguards are still essential to coastal safety, technology-driven support systems can dramatically improve their efficacy by delivering real-time intelligence and predictive insights.

### The purpose of the project

The SeaYou Project seeks to address these issues by creating an IoT-enabled drone and camera system intended exclusively for drowning prevention and coastal rescue. By combining IoT sensors, predictive drift modelling, and drone-assisted search, the system will provide lifeguards and coastguards with actionable intelligence. This includes:

- Real-time swimmer tracking to identify distressed individuals.
- Ocean current prediction modelling for search zone optimisation.
- Drones are deployed to maintain visual contact and provide airborne situational awareness.

SeaYou's motivation originates from three essential drivers:

- **Humanitarian Need:** Preventing drownings immediately saves lives and decreases the emotional and societal costs of coastal disasters.
- **Technological Opportunity:** Advances in IoT platforms (Arduino, Raspberry Pi, ESP32), drone technology, and predictive modelling methods offer a viable path to developing a cost-effective, scalable system.
- **Preventive Safety:** Moving beyond reactive rescue, SeaYou now provides preventive alarms for crowd density and rip currents, allowing lifeguards to respond before crises escalate.

Furthermore, the system is built with climate resilience in mind, providing adaptation to rising sea levels, storm surges, and unexpected weather patterns. SeaYou's future-proofing makes it vital not only for current coastal safety, but also for long-term sustainability.

## Project Literature Review

### Introduction

Drowning remains a critical global public health issue, with the World Health Organization estimating over 236,000 annual fatalities worldwide (WHO, 2021). South Africa, in particular, experiences high drowning rates during peak holiday seasons along crowded coastlines, where traditional lifeguard monitoring is constrained by visibility, response time, and resource allocation. Emerging technologies such as IoT-enabled monitoring, computer vision, and drone-assisted rescue have been proposed to augment human lifeguards and improve coastal safety. This literature review examines current systems, their limitations, and how the *SeaYou* project advances the field.

### Current Approaches to Drowning Prevention

#### Camera-Based AI Detection

Computer vision systems analyse swimmer behaviour to detect distress. For example, Aqua Wings employs AI-powered drowning detection through overhead cameras (Aqua Wings, 2023). These systems are non-invasive and scalable but suffer from reduced accuracy in low-visibility conditions such as fog, storms, or nighttime (George Rescue Tech, 2022).

#### Wearable Sensor Systems

Sensor-based approaches rely on swimmers wearing devices that monitor vitals or motion patterns. Studies highlight their effectiveness in controlled environments, but adoption is limited in public beaches due to practicality and compliance issues (Cygnus Project, 2021). While they provide direct physiological data, they are unsuitable for large-scale coastal monitoring.

#### Drone-Assisted Rescue

Drones have been deployed to deliver flotation devices or maintain visual contact with victims. A case study in George, South Africa, demonstrated drones reducing incident response times significantly (George Rescue Tech, 2022). However, most drone systems are reactive and lack integration with predictive modelling or IoT-driven situational awareness.

#### Hybrid Detection and Rescue Systems

Some systems attempt to combine detection with rescue. The Cygnus system integrates IoT and AI/ML for vision-based drowning detection but has limited rescue capabilities (Cygnus Project, 2021). Similarly, autonomous rescue drones have been tested in Europe, but they remain costly and lack scalability for smaller coastal towns.

## Conclusion

Current drowning prevention systems either focus on detection, rescue or recovery but rarely integrate both effectively. The *SeaYou* project advances the field by merging IoT-enabled monitoring, predictive drift modelling, and drone-assisted rescue into one system. Its emphasis on real-time situational awareness, preventive safety, and climate resilience positions it as a promising innovation for South Africa's high-risk coastlines.

## Non-Functional Requirements

### Performance Targets

- **Real-Time Processing:** The system must provide real-time swimmer tracking and dynamic monitoring updates to ensure situational awareness for rescue teams.
- **Response Efficiency:** A core goal is to improve rescue efficiency by providing immediate search zone recommendations and location data.
- **Transmission Speed:** Use of Wi-Fi and Bluetooth is intended for real-time data transmission between hardware components like cameras, drones, and monitoring dashboards.
- **Uptime and Reliability:** The system is designed to be climate-resilient, maintaining functionality during storm surges, rising sea levels, and unpredictable weather.

### Security Expectations

- **Data Integrity:** Reliable communication protocols must be developed to ensure the accuracy of real-time updates and search recommendations.
- **Legal & Ethical Standards:** The project includes "Legal Research" as a task, which would involve addressing privacy concerns related to facial recognition and 360° camera monitoring.
- **Search and Rescue Privacy:** While the system uses facial recognition and body tracking, its application is strictly for locating victims and managing crowd density for safety.

### Scalability, Usability, and Maintainability

- **Scalability:** The architecture is designed to be customizable and cost-effective, allowing for adoption in both small coastal towns and by large national organisations.

- **Usability:** A major objective is "Objective 4: Confirm the system with real-world testing to evaluate accessibility, functionality, and ease of use" for lifeguards and NSRI teams.
- **Maintainability:** The use of modular IoT platforms like Arduino, Raspberry Pi, and ESP32 allows for iterative improvements and easier component replacement.
- **Interoperability:** The technology stack (React, Power BI, Figma) supports a standard frontend/backend structure for future software updates.

## Compliance

### Regulatory Research:

- **Aviation Authority Compliance:** All drone operations must adhere to Part 101 of the Civil Aviation Regulations (2011) as governed by the South African Civil Aviation Authority (SACAA).
- **RPAS Classification:** Remotely Piloted Aircraft Systems (RPAS) are classified by their impact energy, maximum take-off mass (MTOM), and line-of-sight capability (e.g., Class 1A for MTOM <1.5 kg and height <400 ft).
- **Operational Certification:** An initial approval application must include a manufacturer's UAS Operating Manual and a detailed submission covering technical specifications like flight envelope capability, payload details, and frequency of use.
- **Safety and Emergency Documentation:** The system must document procedures for multiple emergency scenarios, including:
  - Loss of data link or radio control link.
  - Loss of propulsion or engine power.
  - GPS/Global Navigation Satellite System (GNSS) failure.
  - Low battery voltage and remote pilot station power loss
- **Performance and Environmental Limitations:** Documentation must clearly define the drone's performance limits concerning meteorological conditions such as wind, humidity, precipitation, and temperature.
- **Hazard Assessment:** An objective hazard assessment is required, which must identify potential failure conditions and establish management and mitigation strategies for those conditions.

**Standardised Protocols:** Development of standard communication protocols for real-time updates to ensure consistency across different hardware integrations.

## 1. Lawful Processing and POPIA Compliance

In South Africa, "biometric information" (which includes facial recognition data) is classified as **Special Personal Information**.

- **The "Purpose" Rule:** Under POPIA, you must have a specific, explicitly defined, and lawful purpose for collecting this data. While "search and rescue" is a strong justification, you must prove that the facial recognition is *necessary* for that goal and not just a "nice-to-have" feature.
- **Prior Authorisation:** Since you are processing special personal information to identify individuals in public, you may be required to obtain **Prior Authorisation** from the Information Regulator before the system goes live.

## 2. Lack of Informed Consent in Public Spaces

- **The Challenge:** It is impossible to get individual consent from every person on a crowded beach.
- **Regulatory Risk:** Processing special personal information without consent is generally prohibited unless it falls under specific exceptions (e.g., for the "establishment, exercise, or defence of a right or obligation in law").
- **Mitigation:** You would likely need to rely on "Legitimate Interest" or "Public Interest" as a legal ground, but this requires a high burden of proof that the safety benefit outweighs the privacy intrusion.

## 3. Mass Surveillance vs. Target Surveillance

- **Indiscriminate Collection:** Even if the goal is to find one distressed swimmer, your 360° cameras will capture thousands of "bystanders" (data subjects) who are not in danger.
- **Data Minimisation:** POPIA requires you to collect only the minimum amount of data needed. A concern is whether the system can function using "object detection" (identifying a human shape) rather than "facial recognition" (identifying a specific person).

## 4. Surveillance of Vulnerable Groups (Children)

- **High Sensitivity:** Beaches are frequented by children. POPIA provides extra protection for the personal information of children.
- **Ethical Barrier:** Using facial recognition on minors without parental consent is one of the most legally sensitive areas of AI surveillance and could lead to significant liability if the data is mishandled.

## 5. Data Security and Breach Risks

- **The "Honey Pot" Effect:** Storing a database of faces and locations makes your system a high-value target for hackers.

- **Liability:** Section 19 of POPIA mandates strict "technical and organisational measures" to prevent loss or unauthorised access. A leak of biometric data is considered a severe breach and can result in fines up to R10 million or imprisonment.

## 6. Function Creep

- **The Concern:** A system designed for "Rescue" could easily be used for "Policing" (e.g., identifying people drinking alcohol on the beach or monitoring protests).
- **Ethical Impact:** This leads to a "chilling effect" where citizens feel they cannot enjoy public spaces without being tracked by the state or private entities.

## Functional Requirements

### IoT-Enabled Monitoring System

- **Camera-based swimmer tracking:** The system must detect and track swimmers in real time using IoT-enabled cameras.
- **360° camera integration:** Cameras must provide zoom and panoramic views for lifeguards.
- **Wireless communication protocols:** The system must transmit data via Wi-Fi/Bluetooth to lifeguards' dashboards.

### Drone Deployment & Rescue Support

- **Drone integration:** Drones must be able to launch automatically when a swimmer is detected in distress.
- **Flotation device delivery:** Drones must carry and drop flotation devices to victims. (Possible integration)
- **Live drone visuals:** Drones must stream real-time video to lifeguards for situational awareness.

### Predictive Drift Modelling

- **Ocean current data analysis:** Algorithms must process ocean current data to predict drift paths of victims.
- **Search zone recommendations:** The system must generate dynamic search zones for rescuers.

### Real-Time Feedback & Alerts

- **Dynamic monitoring updates:** The system must continuously update swimmer location and drift predictions.
- **Crowd density alerts:** Must notify lifeguards when swimmer density exceeds safe thresholds.

- **Rip current alerts:** Must detect and warn about dangerous currents before incidents occur.

## 5. User Interface & Usability

- **Monitoring dashboard:** Must provide lifeguards with a clear, real-time interface (built with Figma/Sketch).
- **Usability testing:** The system must undergo trials with NSRI teams to validate ease of use and effectiveness.

## Technical Constraints

### 1. Hardware Constraints

The drone must include:

- An onboard computing unit capable of running AI inference
- A camera system for real-time ocean monitoring

The onboard system must:

- Support real-time video processing
- Handle continuous operation during flight

Constraint: The onboard hardware has limited processing power and memory, requiring efficient and optimized software.

### 2. Camera Constraints

The drone must be equipped with a high-resolution camera for detecting people in the ocean.

Requirements:

- Minimum 720p resolution
- Real-time video capture (15–30 FPS)

The camera must operate effectively under:

- Bright sunlight
- Water reflections and glare
- Constant motion (waves and wind)

Constraint: Higher resolution improves detection accuracy but increases computational load.

### **3. Operating System Constraints**

The onboard system must run a Linux-based operating system.

Software must be compatible with:

- Embedded systems
- ARM or similar architectures

The OS must support:

- AI inference frameworks
- Real-time processing

### **4. Software Constraints**

The onboard software must perform:

- Video capture and preprocessing
- AI-based person detection
- Drowning behaviour analysis
- Decision-making (alert generation)
- Communication with external systems

Constraints:

- Must be optimized for real-time performance
- Must minimize:
  - CPU usage
  - Memory usage
- Must avoid heavy or non-optimized libraries

### **5. AI Model Constraints**

- AI must run onboard the drone (edge computing).
- The model must:
  - Detect humans in water
  - Identify possible drowning behaviour

Constraints:

- Model must be lightweight and efficient
- Inference time must be:
  - $\leq 500$  ms per frame
- Model must be optimized using:
  - Quantization (e.g., INT8)
  - Reduced input resolution
- Must use lightweight frameworks such as:
  - TensorFlow Lite
  - ONNX Runtime

## 6. API Constraints

Drone → Server

The drone must send:

- Alerts (e.g., drowning detected)
- GPS coordinates
- System status

Example endpoints:

- POST /alert
- POST /telemetry

Server → Drone

The system must support:

- Control commands (e.g., return home, land)
- Configuration updates

Example endpoints:

- POST /command

Constraints:

- APIs must be:
  - Low latency

- Lightweight (JSON format)
- Must function under limited bandwidth conditions

## **7. Communication Constraints**

- The system must support wireless communication:
  - Wi-Fi and/or LTE/4G/5G
- Must ensure:
  - Reliable data transmission
  - Low latency for alerts
- Constraint: Bandwidth is limited, so continuous video streaming should be minimized.

## **8. Performance Constraints**

- The full detection pipeline (capture → inference → decision) must:
  - Complete within 500 ms
- System must operate in near real-time
- Frame processing rate may be reduced to maintain stability

## **9. Environmental Constraints**

- The drone must operate in:
  - Marine environments (saltwater exposure)
  - High wind conditions
  - Direct sunlight
- Hardware must be:
  - Weather-resistant
  - Corrosion-resistant
  - UV-resistant

## **10. Thermal Constraints**

- Continuous AI processing generates heat.
- The system must:
  - Monitor internal temperature
  - Prevent overheating
- Safety mechanisms must:
  - Reduce processing load when needed
  - Trigger safe landing if critical temperature is reached

## **11. Reliability Constraints**

- The system must:
  - Continue operating without constant server connectivity
  - Handle temporary signal loss
- AI must:
  - Minimize false positives (false drowning alerts)
  - Minimize false negatives (missed detections)

### **Data Requirements:**

This project will make use of necessary data collection and manipulation. As per the question of sensitive data being captured by drones and camera the video data will be wiped each day to maintain the beach goers privacy and such only certain data will be captured and send to the database for government statistics purposes.

## User Interface Skeleton

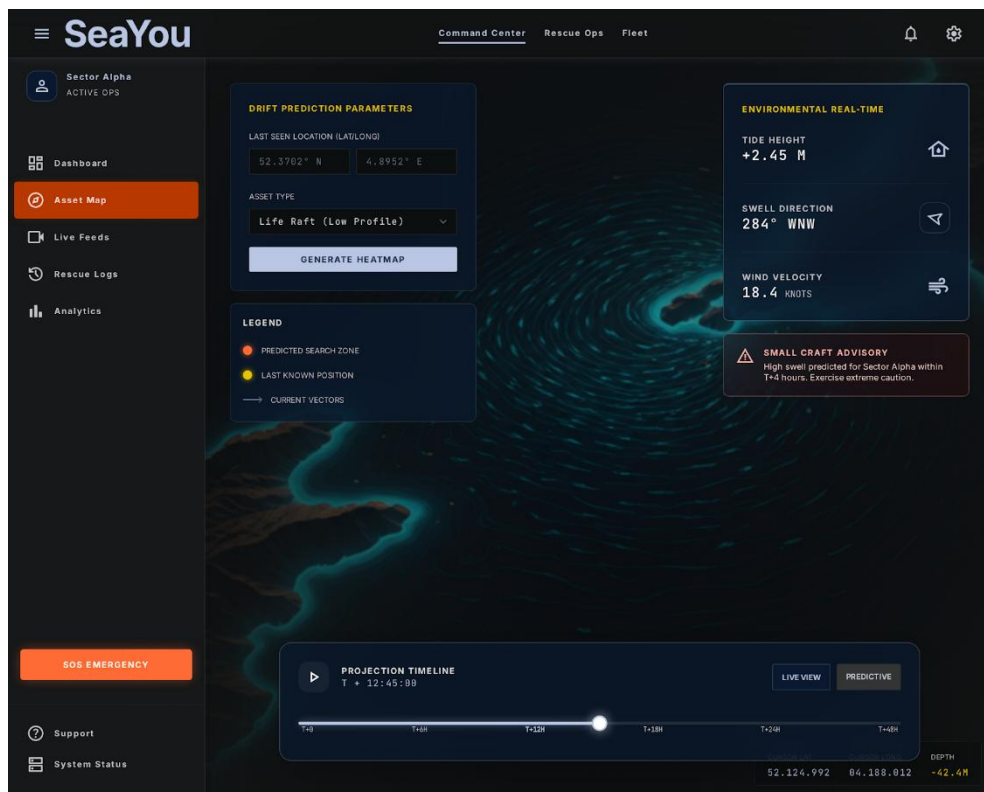


Figure 1.1 – Map/Overview of the location

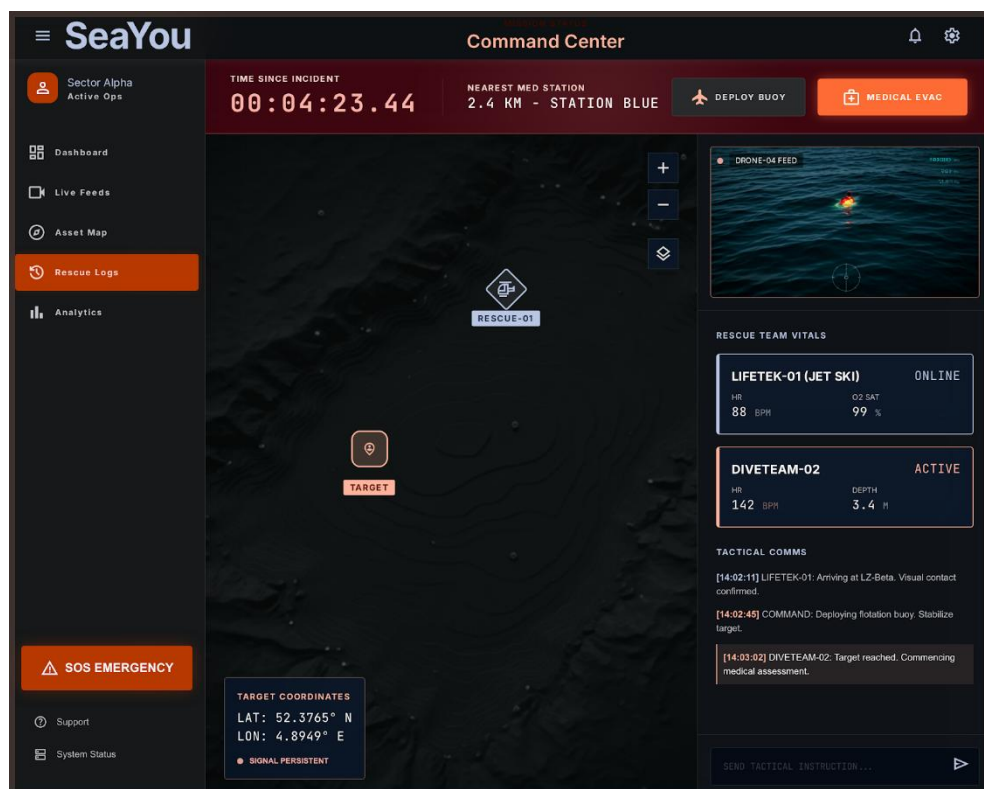


Figure 1.2 – Active Rescue Screen

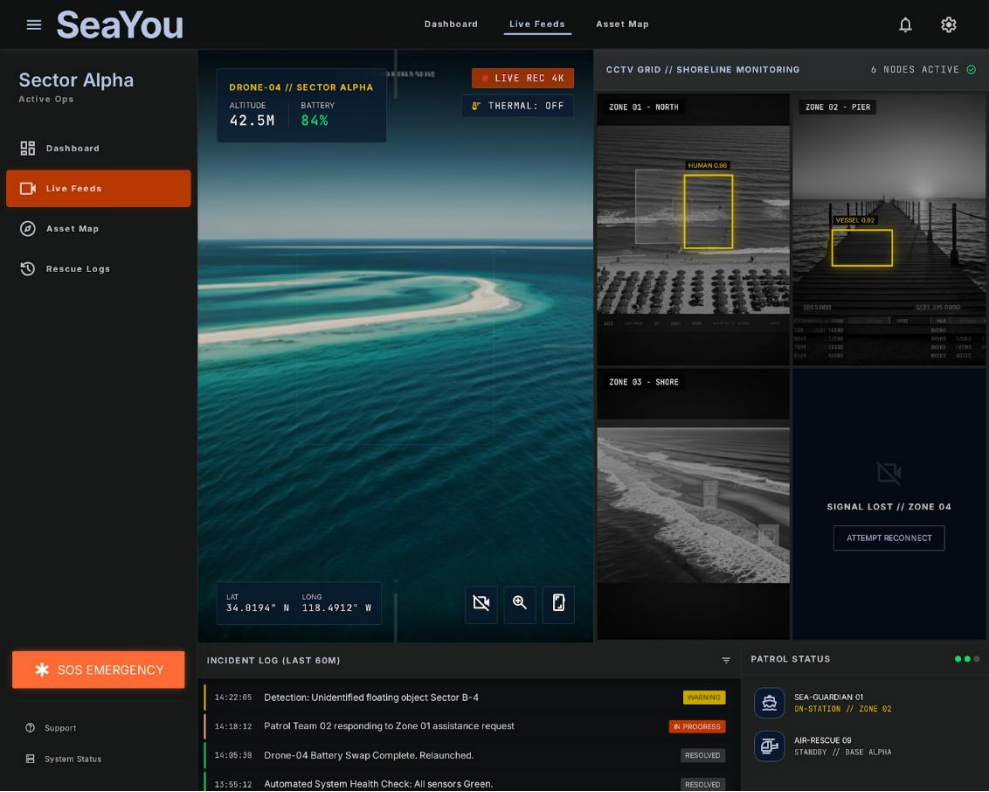


Figure 1.3 – Live feed of drones & Cameras

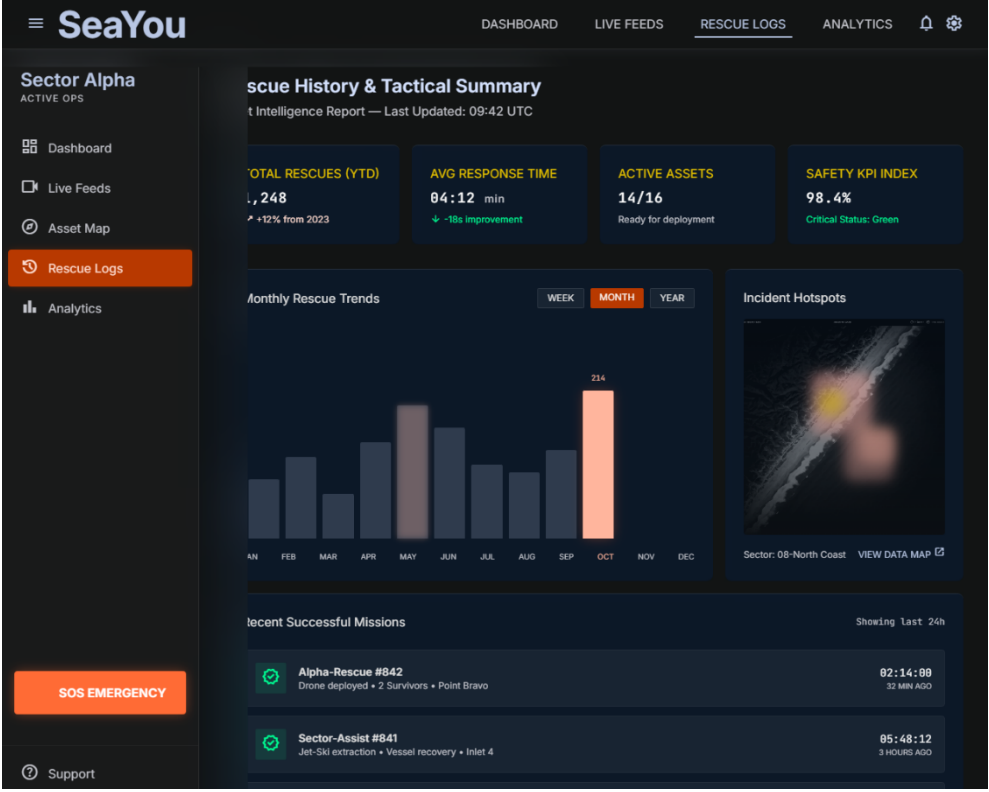


Figure 1.4 – Statistics/Logs for detailed reports

## Data Architecture Specification

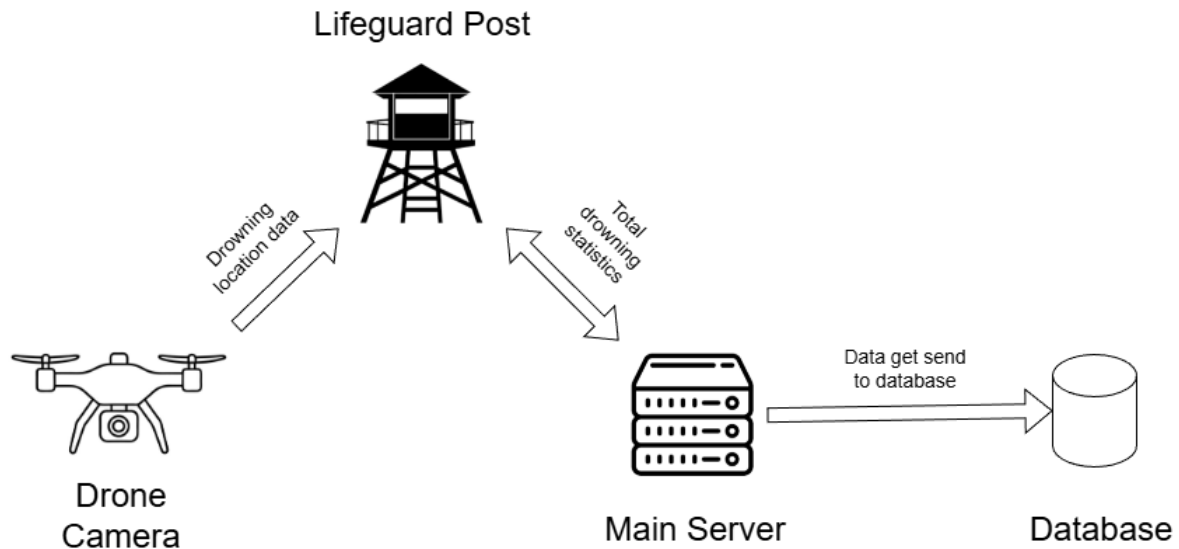


Figure 1: Data Map Diagram

The above diagram indicates the flow of data. Data is sent from the drone and camera on the beach front to the lifeguard post. The data that is sent is locations and alerts for individuals that are drowning. This data is then compiled in a list and send to the main server so that it can be stored on a database. Other data that is sent from the Lifeguard post to the main server is total individuals who have drowned and the total drowned victims that were recovered. This data must be collected for the South African Medical Research Council, and this data will also be used to determine possible hot spots in coastal waters, and this can be used to possibly reduce drowning in coastal waters.

Tables start on next page...

## Data:

The following tables indicate the type of data that will be collected the format as well as the volume of this data.

| Drone — Data Table     |                   |                          |                          |                                       |
|------------------------|-------------------|--------------------------|--------------------------|---------------------------------------|
| Data Field             | Data Type         | Format                   | Sample Value             | Estimated Volume                      |
| GPS latitude           | Float             | Decimal degrees (WGS84)  | -33.924868               | 1 record / 500 ms → ~7,000 records/hr |
| GPS longitude          | Float             | Decimal degrees (WGS84)  | 18.424055                | 1 record / 500 ms → ~7,000 records/hr |
| Altitude (AGL)         | Float             | Metres, 2 decimal places | 25.40 m                  | 1 record / 500 ms                     |
| Heading                | Float             | Degrees (0–360)          | 187.3°                   | 1 record / 500 ms                     |
| Flight speed           | Float             | m/s, 2 decimal places    | 4.20 m/s                 | 1 record / 500 ms                     |
| Battery level          | Integer           | Percentage (0–100)       | 72%                      | 1 record / 10 s                       |
| AI detection flag      | Boolean           | True / False             | True                     | 1 per inference (every 200 ms)        |
| Bounding box           | Array of integers | JSON [x, y, w, h] pixels | [420, 310, 85, 95]       | 1 per positive detection              |
| Timestamp (UTC)        | DateTime          | ISO 8601 (ms precision)  | 2025-04-08T14:32:01.452Z | Embedded in every record              |
| Drone ID               | String            | UUID v4                  | drone-a3f9-4d21-bc87     | Static per session                    |
| Signal strength (RSSI) | Integer           | dBm                      | -67 dBm                  | 1 record / 1 s                        |

| Lifeguard Post — Data Table |               |                                     |                                                     |                                             |
|-----------------------------|---------------|-------------------------------------|-----------------------------------------------------|---------------------------------------------|
| Data Field                  | Data Type     | Format                              | Sample Value                                        | Estimated Volume                            |
| Victim live GPS pin         | Float pair    | GeoJSON Point                       | <code>{"type":"Point","coords":[-33.9,18.4]}</code> | Updated every 2 s during incident           |
| Victim Image Report         | Binary        | JPEG 480p                           | frame_thumb.jpg (~40 KB)                            | 1 pushed per detection; refreshes every 5 s |
| Rescue drone live position  | Float pair    | GeoJSON Point                       | <code>{"lat":-33.922,"lon":18.426}</code>           | Updated every 2 s                           |
| Post ID                     | String        | Alphanumeric                        | POST-SB-04                                          | Static config value                         |
| Lifeguard on-duty ID        | String        | Staff alphanumeric ID               | LG-2041                                             | Set at shift start                          |
| Shift log entry             | String / JSON | UTF-8 free text + structured fields | <code>{action, lg_id, ts, notes}</code>             | 1 per operator action (~500 B)              |
| Map tile data (beach chart) | Binary tiles  | PNG / WebP map tiles (OSM/custom)   | tile_z14_x8_y10.png                                 | Cached; ~200 KB initial load                |

| Main Server — Data Table     |            |                      |                                           |                               |
|------------------------------|------------|----------------------|-------------------------------------------|-------------------------------|
| Data Field                   | Data Type  | Format               | Sample Value                              | Estimated Volume              |
| Incident ID                  | String     | UUID v4              | inc-7f3a-4c12-9e01                        | 1 per drowning event detected |
| Victim GPS (lat/lon)         | Float pair | Decimal degrees JSON | <code>{"lat":-33.924,"lon":18.424}</code> | 1 per incident                |
| Detection timestamp          | DateTime   | ISO 8601 (ms)        | 2025-04-08T14:32:01.452Z                  | 1 per incident                |
| Assigned rescue drone ID     | String     | UUID v4              | drone-rescue-b12c                         | 1 per incident, may update    |
| Response time estimate (ETA) | Integer    | Seconds              | 47 s                                      | Updated every 5 s             |
| Lifeguard post ID            | String     | Alphanumeric         | POST-SB-04                                | 1 per incident                |
| Number of victims (count)    | Integer    | Whole number         | 1                                         | Updated per re-inference      |

| Database - Data Table      |                 |                                              |                                        |                                    |
|----------------------------|-----------------|----------------------------------------------|----------------------------------------|------------------------------------|
| Data Field                 | Data Type       | Format                                       | Sample Value                           | Estimated Volume                   |
| Incident record            | JSON document   | Structured JSON (MongoDB / PostgreSQL JSONB) | {incident_id, timestamps, coords, ...} | ~2 KB/incident                     |
| Full drone telemetry log   | Time-series     | CSV or InfluxDB line protocol                | ts,lat,lon,alt,hdg,bat,...             | ~50 MB/drone/day                   |
| Rescue outcome status      | Enum (string)   | RESCUED / DECEASED                           | RESCUED                                | 1 per incident; updated post-event |
| Rescue drone telemetry log | Time-series     | CSV or InfluxDB line protocol                | ts,lat,lon,alt,speed,...               | ~25 MB/rescue mission              |
| AI model version           | String          | SemVer                                       | v2.4.1                                 | 1 per incident                     |
| Audit trail (event log)    | JSON array      | Timestamped action log                       | [{action, actor, ts}, ...]             | ~5 KB/incident                     |
| Daily incident summary     | Aggregated JSON | JSON (counts, avg response time)             | {date, total, avg_resp_s}              | ~1 KB/day/beach                    |
| Model performance metrics  | JSON            | Precision, recall, F1 per day                | {precision:0.97, recall:0.93}          | ~1 KB/day                          |

## Testing & Validation Criteria

The SeaYou system will require rigorous testing to ensure reliability, accuracy, and usability in real-world coastal rescue scenarios. The following criteria define how the system will be validated:

### 1. Functional Testing

**Purpose:** Verify that all system components operate as intended.

**Criteria:**

- IoT cameras successfully detect and track swimmers in real time.
- Drone deployment is triggered correctly based on system alerts.
- Communication between devices (Central Control Station, Drones and Computational Centre) is stable.

- Flotation device release mechanism functions accurately.

**Validation Method:**

- Simulated drowning scenarios in controlled environments.
- Unit and integration testing of hardware and software components.

## **2. Real-Time Performance Testing**

**Purpose:** Ensure the system meets real-time operational requirements.

**Criteria:**

- Latency of live video feed  $\leq 2-3$  seconds.
- Data transmission delays remain minimal under load.
- Alerts are generated instantly when a risk is detected.

**Validation Method:**

- Stress testing with multiple simultaneous swimmers.
- Network performance testing under different bandwidth conditions.

## **3. Predictive Model Accuracy Testing**

**Purpose:** Validate the drift prediction algorithm.

**Criteria:**

- Drift prediction accuracy within an acceptable error margin (e.g.,  $\pm 5-10$  meters depending on conditions).
- Consistency of predictions across different ocean conditions.

**Validation Method:**

- Compare predicted drift paths with real-world float tracking experiments.
- Use historical ocean current datasets for validation.

## **4. Environmental & Robustness Testing**

**Purpose:** Ensure system reliability in harsh coastal conditions.

**Criteria:**

- System operates effectively in:
  - Low visibility (fog, rain, night)
  - High wind conditions

- Saltwater exposure
- Thermal/infrared sensors accurately detect humans in low visibility.

**Validation Method:**

- Field testing at beaches during different weather conditions ( If possible otherwise will need to be testing in near by dams for initial concept proving ).
- Controlled environmental simulations where possible.

**5. Usability Testing**

**Purpose:** Evaluate ease of use for lifeguards and rescue personnel.

**Criteria:**

- Interface is intuitive and easy to interpret.
- Minimal training required for effective use.
- High user satisfaction (measured via surveys).

**Validation Method:**

- Testing sessions with lifeguards and **NSRI teams**.
- Collection of feedback through questionnaires and interviews post deployment.

**6. Reliability & Availability Testing**

**Purpose:** Ensure system consistency and uptime.

**Criteria:**

- System uptime  $\geq 95\%$  during operation.
- Minimal system crashes or failures.
- Quick recovery from faults.

**Validation Method:**

- Long-duration testing (soak testing).
- Failure injection testing to assess recovery.

**7. Drone Operation & Safety Testing**

**Purpose:** Validate safe and effective drone usage.

**Criteria:**

- Accurate navigation to target coordinates.

- Safe deployment of flotation devices.
- Collision avoidance functioning correctly.
- Quick response and drone deployment times

**Validation Method:**

- Controlled drone flight tests.
- Scenario-based rescue simulations.

**8. Scalability Testing**

**Purpose:** Ensure the system can scale for broader deployment.

**Criteria:**

- System performance remains stable with increased users/devices.
- Efficient handling of multiple simultaneous rescue scenarios.

**Validation Method:**

- Load testing with multiple IoT devices and drones.
- Simulation of high-density beach scenarios.

**9. Validation Metrics Summary**

| Aspect             | Metric                     | Target                  |
|--------------------|----------------------------|-------------------------|
| Response Time      | Alert latency              | ≤ 3 seconds             |
| Tracking Accuracy  | Swimmer detection accuracy | ≥ 90%                   |
| Drift Prediction   | Position error             | ±5–10 meters            |
| System Reliability | Uptime                     | ≥ 95%                   |
| Usability          | User satisfaction score    | ≥ 80% positive feedback |
| Drone Accuracy     | Target delivery precision  | ≤ 2 meters error        |

**10. Final Validation Approach**

The system will follow a **multi-phase validation process**:

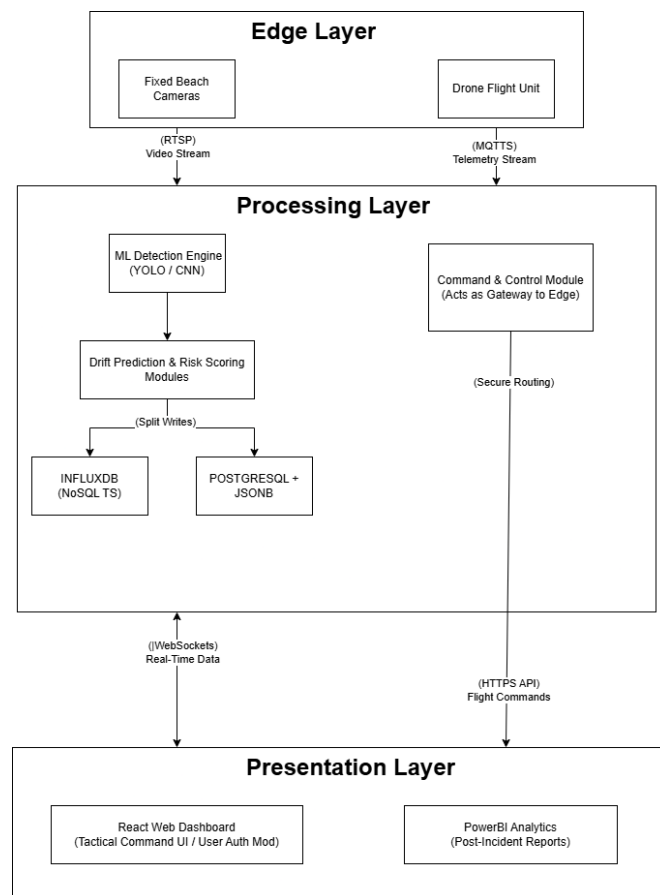
1. **Alpha Testing** – Controlled lab and simulated environments, possible dam usage for checking if the drowning recognition software works properly
2. **Beta Testing** – Real-world beach trials with lifeguards
3. **User Acceptance Testing (UAT)** – Final validation by stakeholders (lifeguards and public that are on the beach)

## Step 1: Define the System Type

SeaYou is a drone-based and distributed IoT real-time monitoring system. It is not a traditional web app or mobile app, but a multi-layer system with physical hardware, a processing server in the back-end, and a web-based interface collaborating as a single platform.

## System Architecture and Design

### System Overview



## Brief Interaction Flow

- Step 1 (Ingestion): Fixed beach cameras and drones stream live footage to the backend processing layer over RTSP, while the drone continually broadcasts its flight telemetry over MQTTS.
- Step 2 (Analysis): The backend processes the video in-memory through the ML Detection Engine. If a drowning risk is found, coordinates are fed into the Drift Prediction Engine.
- Step 3 (Alerting): The Risk Scoring module flags a breach, saves the event details, and pushes an immediate warning down to the React Web Dashboard via WebSockets.
- Step 4 (Command & Intervene): The lifeguard logs into the dashboard, views the live feed, and clicks "DEPLOY FLOTATION". This request is sent via HTTPS to the central backend, which validates the credentials and forwards the signed actuation command down to the drone's ESP32 microcontroller to release the life jacket.

## Component Description

### Frontend / UI Layer

- React Web Dashboard: The mission-critical interface for lifeguards. Designed as a high-contrast tactical command centre featuring real-time optical/thermal video streams, live drone battery metrics, automated alert pop-ups, and interactive map layers with glowing drift vectors.
- PowerBI Analytics Panel: A secondary, historical interface used by beach managers and regional administrators to review post-incident reports, evaluate response timelines, and observe long-term safety trends.

### Backend / Application Logic

- Central Processing Server: Built using Python and Node.js microservices to coordinate ingestion pipelines, manage websocket rooms, interface with underlying machine learning scripts, and safely route commands from the UI back down to the field hardware.
- User Auth Module: An isolated application security gateway positioned upstream of the interfaces to handle user login, issue session tokens, and verify that incoming endpoint requests match specific roles.

## Database / Storage

- InfluxDB (Time-Series NoSQL): Dedicated exclusively to capturing high-frequency, append-only flight telemetry streams from the drones. It isolates sub-second updates away from transactional tables to avoid performance lags.
- PostgreSQL with Native JSONB (Relational Core): Houses structural entities requiring rigid transactional accuracy (user data, device registries, audit history) while leveraging dynamic JSONB columns to capture flexible, semi-structured incident metadata documents.

## APIs or External Services

- OpenDrift API Core: An external/locally containerised Lagrangian trajectory framework combined with meteorological feeds to calculate dynamic victim drift predictions across ocean currents.

## Technology Stack

| Layer           | Proposed Technologies                                                                    |
|-----------------|------------------------------------------------------------------------------------------|
| Frontend / UI   | HTML5, CSS3, JavaScript, React.js, Tailwind CSS                                          |
| Backend / Logic | Python (FastAPI / Flask for ML services), Node.js (Express for API routing & WebSockets) |
| Database        | PostgreSQL (v15+ with JSONB support), InfluxDB (v2.x Time-Series Engine)                 |
| Dev Tools / IDE | VS Code, Arduino IDE, PyCharm                                                            |
| Version Control | Git + GitHub                                                                             |
| Deployment      | Docker Containers, Localised Host Servers (Belgium Campus environment)                   |

## Data Design

The SeaYou system implements a relational schema to manage structural integrity and transactional states, combined with a dedicated time-series stream for volatile edge data.

### Core Entities and Their Attributes

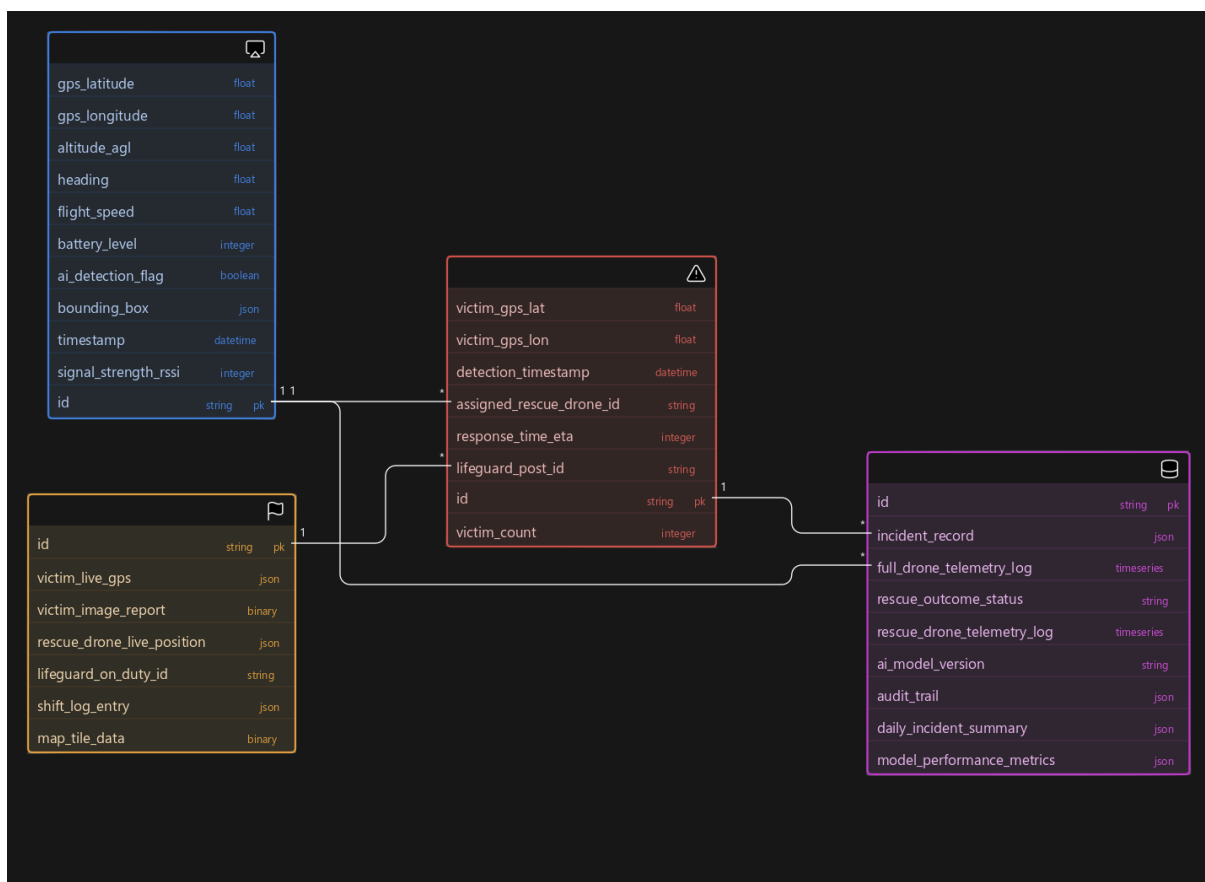
Based on the system database structure, the primary entities include:

- **Users:** Manages platform operator identities.
  - *Attributes:* UserID (PK), Username, PasswordHash, Email, RoleID (FK), CreatedAt.
- **UserRoles:** Defines structural permission levels.
  - *Attributes:* RoleID (PK), RoleName (e.g., *Lifeguard*, *Rescue Team*, *Admin*), Description.
- **Equipment:** Tracks physical IoT assets actively deployed on the shoreline.
  - *Attributes:* EquipmentID (PK), Name, SerialNumber, Status (e.g., *Active*, *Maintenance*, *Low Battery*), EquipmentTypeID (FK), LastMaintenanceDate.
- **EquipmentTypes:** Differentiates edge hardware profiles.
  - *Attributes:* EquipmentTypeID (PK), TypeName (e.g., *Quadcopter Drone*, *Fixed Camera Array*), Description.
- **Incidents:** Logs unique drowning or distress events flagged by the ML Detection Engine.
  - *Attributes:* IncidentID (PK), EquipmentID (FK - asset that detected it), Location (Point coordinates/GPS pair), StartTime, EndTime, SeverityLevel (Risk score evaluation), Status (Enum: *Pending*, *Resolved*, *False Alarm*).
- **RescueActions:** Captures chronological tactical operations executed during an active emergency lifecycle.
  - *Attributes:* ActionID (PK), IncidentID (FK), ActionType (e.g., *Drone Deployment*, *Flotation Dropped*, *Manual Dispatch*), Timestamp, PerformedBy (FK to Users), Notes.

### Relationships Between Entities

- **One-to-Many (UserRoles to Users):** A single defined security role can be assigned to multiple users, while each user possesses exactly one primary role.

- One-to-Many (EquipmentTypes to Equipment): A type profile covers multiple physical hardware devices deployed across different beach posts.
- One-to-Many (Equipment to Incidents): A single camera or drone can detect or be assigned to multiple independent incidents over its operational lifespan.
- One-to-Many (Incidents to RescueActions): An emergency incident lifecycle acts as a parent log tracking an ongoing sequence of multiple distinct rescue actions.
- One-to-Many (Users to RescueActions): A lifeguard or rescue team operator can execute and sign off on multiple tactical rescue actions.



## Data Flow Implementation

1. Authentication Path: When an operator attempts a dashboard login, the User Auth Module queries Users and checks the linked UserRoles to verify authorisation levels before unlocking UI modules.
2. Detection and Logging Path: When the backend ML Detection Engine identifies distress from an RTSP camera stream, it inserts a new row into Incidents linked directly to that camera's EquipmentID, shifting the alert panel on the React UI to high priority.

3. Command and Intervention Path: When a lifeguard triggers a drone dispatch or float drop, a transaction is committed to RescueActions with the corresponding UserID, IncidentID, and timestamp, maintaining a legally compliant audit log under POPIA safeguards before signals are sent to the edge microcontrollers.
4. Telemetry Streaming Path (Off-Schema): High-frequency flight data points continue to stream into InfluxDB tagged with the matching EquipmentID, ensuring sub-second location updates don't overwhelm these relational tables.

## **Security Considerations**

### Authentication

The User Auth Module uses email/username and hashed passwords managed on the processing server. Secure access tokens (JSON Web Tokens) are issued upon verification, meaning the React web dashboard must pass a signed, unexpired bearer token with every HTTP request.

### Authorisation (Role-Based Access Control)

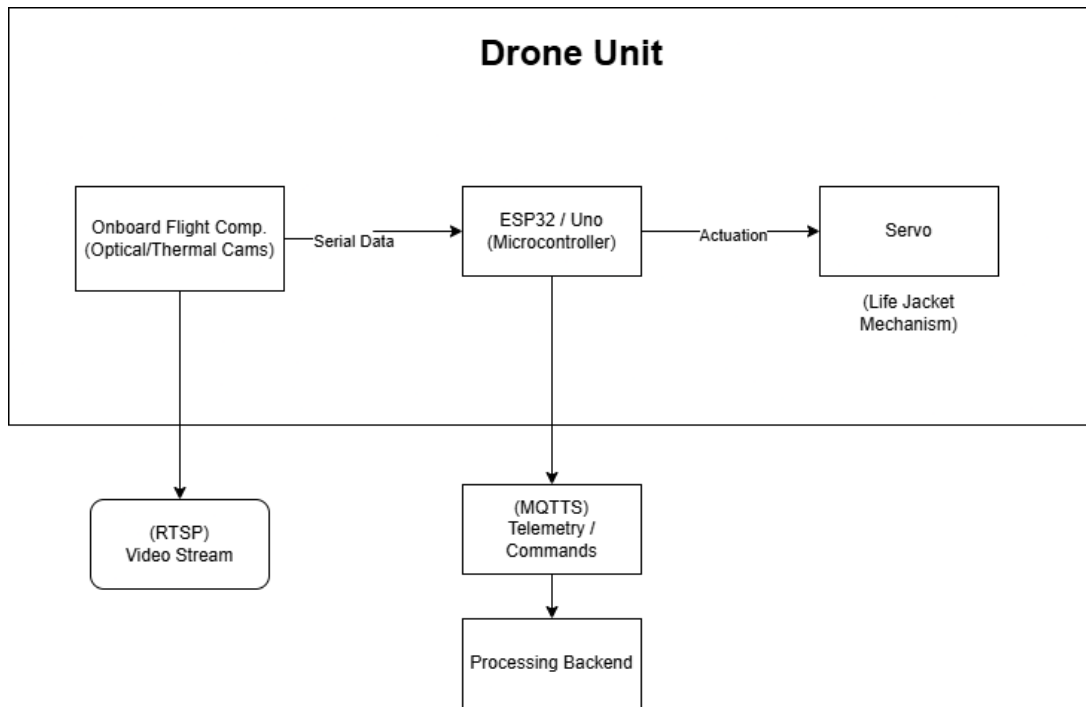
The application enforces strict structural permission boundaries:

- Lifeguard: Authorised to read warning panels, monitor active video feeds, log shift changes, and click the software execution triggers to deploy localised drone flight components.
- Rescue Team: Authorised to read structural search zones, view drift coordinates, and track macro beach status boards during cross-agency operations.
- Admin: Complete structural permissions to add/modify user accounts, alter device registration links, check container performance logs, and perform hardware configuration tasks.

## **Hardware Design**

### Platform Overview

The physical system uses a distributed hardware ecosystem composed of fixed high-definition shore cameras, custom quadcopter drone chassis units, and specialised onboard microcontrollers (ESP32 / Arduino Uno frameworks) handling localised telemetry transmission and physical payload mechanisms.



### Key Components

- **Sensors:** Custom dual-payload optical and thermal/infrared sensor pods mounted to the drone frame to allow effective tracking during low-visibility or night-time emergencies.
- **Actuators (Life Jacket Mechanism):** A highly responsive servo-motor mechanism mounted to the drone frame, wired to an Arduino interface that triggers a physical drop hook to deploy an inflatable life jacket upon command.
- **Connectivity:** High-gain dual-band Wi-Fi and long-range cellular network antennas attached to the mobile platforms to guarantee sustained network connectivity across active search and rescue corridors.

### System Integration & Constraints

The physical components connect with the software backend using standardised communication protocols. The onboard computer acts as an edge router, encoding video to stream via RTSP while parsing internal serial data strings (GPS coordinate matrices, battery percentages) to pass up to the backend server through the ESP32's MQTTS channel.

Flight operations adhere closely to SACAA Part 101 Aviation Safety rules. Automated safety loops are hardcoded directly into the edge microcontroller: if the connection link to the processing backend drops, the system bypasses cloud inputs and initiates an

automatic *Return to Home (RTH)* sequence or a safe landing routine away from active swimmer coordinates.

## **Algorithms and Computational Methods**

### Machine Learning & Prediction Frameworks

1. Object Detection Engine (YOLO / Convolutional Neural Networks):
  - *Problem Solved:* Automated computer vision identification of human swimmers showing active flailing or distress signatures within wide-area aquatic frames.
  - *Rationale:* YOLO offers exceptional inference performance, allowing processing frames to run at high frame rates on host server engines to provide near-instantaneous emergency identification.
  - *Assumptions:* Requires a diverse training baseline of aerial coastal imagery, thermal profiles, and water disturbance patterns to avoid false alarms caused by waves or watercraft.
2. Drift Prediction Engine (LSTM Networks & OpenDrift):
  - *Problem Solved:* Projecting the forward movement and drift pathway of a victim caught within moving rip currents.
  - *Rationale:* Long Short-Term Memory (LSTM) networks excel at identifying sequential patterns over time, mapping changing current vectors against real-time variables to optimise emergency search grids.

## **Data Analytics Approach**

### Data Preprocessing

Incoming operational parameters pass through real-time formatting channels before long-term storage. Telemetry packets are scrubbed for structural anomalies, duplicate spatial updates are dropped, and timestamp vectors are synchronised to Coordinated Universal Time (UTC) to guarantee structural alignment between recorded flight lines and incident logs.

### Analytical & Visualisation Techniques

- **Descriptive Statistics:** Compiles crucial performance benchmarks, including mean response intervals, average drone battery consumption rates per deployment, and incident concentrations across different beach zones.

- **Spatial Density Modelling:** The system aggregates historic incident coordinates to render highly accurate coastal risk heatmaps within the presentation interface.
- **PowerBI Dashboards:** Generates interactive post-incident performance matrix dashboards, giving administrators visual tools to identify high-risk seasonal zones and refine lifeguard resource allocation.

Alignment with Scope

The structural design of this architecture connects directly with the in-scope deliverables established in your SeaYou project documentation:

- **Real-Time Tracking & Emergency Alerting:** Achieved by coupling the high-frequency capabilities of the InfluxDB engine with low-latency WebSocket pathways routing up to the React Web Dashboard.
- **Autonomous Flotation Delivery:** Supported by isolating the hardware control logic into a secure backend command mechanism that safely links presentation triggers to edge Arduino controllers.
- **Regulatory & Privacy Compliance:** Addressed by limiting data retention profiles, establishing clear role boundaries, and designing the ML pipeline for object tracking rather than facial indexing, directly aligning the system with POPIA and SACAA Part 101 legal constraints.

Main Modules/Sections

The system is divided into eleven major modules spread across the three layers. Each module has a single, clearly defined responsibility.

Edge Layer

| Module                | Responsibility                                                                                                       |
|-----------------------|----------------------------------------------------------------------------------------------------------------------|
| Drone Unit            | Mobile aerial unit with optical and thermal cameras; performs edge AI inference and deploys life jackets via Arduino |
| Beach Camera Array    | Fixed cameras overlooking the shoreline for continuous wide-area monitoring and rip current detection                |
| Life Jacket Mechanism | Arduino-controlled actuator that physically releases a flotation device on command from the server                   |

## Processing Layer

| Module                  | Responsibility                                                                                                  |
|-------------------------|-----------------------------------------------------------------------------------------------------------------|
| ML Detection Engine     | Runs YOLO swimmer detection and CNN person-in-water models on incoming video frames                             |
| Drift Prediction Engine | Uses LSTM networks and OpenDrift to forecast where a victim will drift based on ocean currents and tides        |
| Risk Scoring and Alerts | Evaluates detection confidence against risk thresholds and triggers distress alerts when confirmed              |
| Command and Control     | Sends instructions back to the ESP32/Arduino — navigation coordinates, life jacket release commands             |
| Database                | Stores incident records, drone telemetry, rescue outcomes, and model performance metrics (PostgreSQL, InfluxDB) |

## Presentation Layer

| Module            | Responsibility                                                                                              |
|-------------------|-------------------------------------------------------------------------------------------------------------|
| React Dashboard   | Primary lifeguard interface showing live video, GPS tracking, drift predictions, alerts, and drone controls |
| PowerBI Analytics | Secondary reporting interface querying historical incident data to identify high-risk zones and trends      |
| User Auth Module  | Manages role-based access for Lifeguard, Rescue Team, and System Admin users                                |

## How the Parts Connect

The three layers are used mainly to transmit data downwards, with a significant reverse path being hardware commands.

The sequence of connections is shown as follows:

1. The Beach Camera Array and the Drone Unit record live video and send it to the Backend Server via RTSP using WiFi.
2. The ML Detection Engine and Drift Prediction Engine analyze the incoming data and send the results to the Risk Scoring module.
3. When a risk threshold is reached, the Risk Scoring module will provide an alert and request the Command and Control module to take action.
4. The Command and Control module transmits down to the ESP32/Arduino on the drone (life jacket release, navigation) to deploy it.
5. The React Dashboard receives processed alerts, GPS and drift forecasts, which are displayed in real time to the lifeguard.
6. The Database is where all the incident data are written and then the PowerBI Analytics query the Database to provide historical reporting.

Both the React Dashboard and the PowerBI Analytics rely on the Processing Layer - there is no direct communication with the Edge Layer hardware. User Auth Module is placed before both presentation components that regulate access on a role basis.

#### Hardware List

- **Drone Unit**
  - Optical cameras (HD)
  - Thermal cameras (IF POSSIBLE)
  - Arduino and ESP32 microcontrollers
  - Raspberry Pi 4-8G ram
  - Life jacket release actuator
- **Beach Camera Array**
  - Fixed shoreline cameras (wide-angle)
  - Wi-Fi/RTSP streaming capability
- **Life Jacket Mechanism**
  - Arduino-controlled actuator system

#### Software List (Processing + Presentation Layers)

- **Backend / Processing Layer**
  - YOLO (object detection for swimmers)
  - CNN (Detection for people in water)

- LSTM (drift prediction)
- OpenDrift (ocean current modeling)
- Risk Scoring & Alerts module
- Command & Control module (ESP32/Arduino integration)
- Databases: PostgreSQL + InfluxDB
- **Presentation Layer**
  - React Dashboard (real-time monitoring interface)
  - PowerBI (analytics and historical reporting)
  - User Authentication module (role-based access control)

## **Planned User Interactions**

### **1. Lifeguard**

- Monitors swimmers using the system dashboard.
- Receives alerts and responds to emergencies.
- Deploys drones and tracks victims.

### **2. Rescue Team**

- Receives location and ocean current predictions.
- Coordinates rescue operations.
- Uses system data for decision-making.

### **3. System Admin**

- Manages system settings and user access.
- Monitors system performance and updates.
- Make sure all the IoT devices (cameras, drones, sensors) gets serviced and stays up to date.

### **4. Public User / Guest**

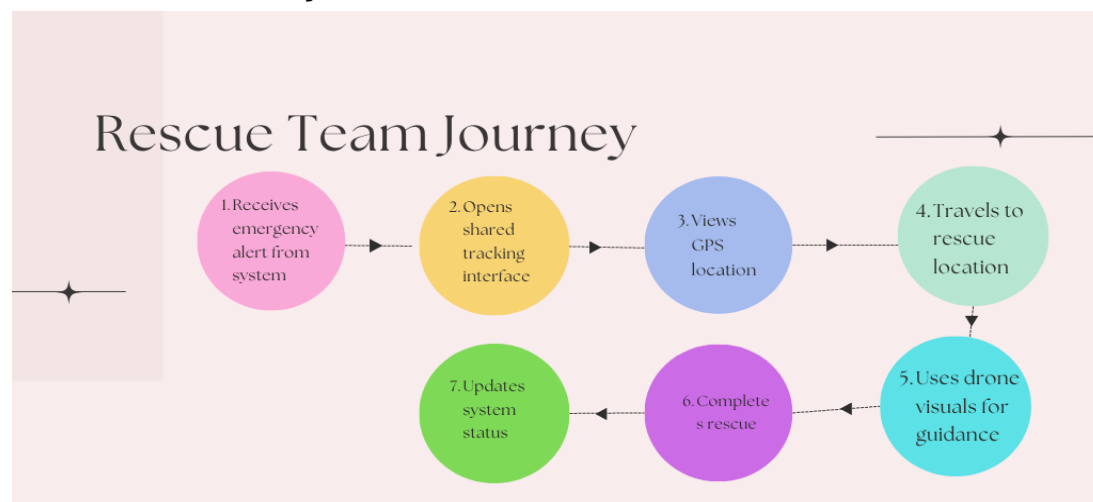
- Receives safety alerts (rip currents, crowd density).
- Does not interact directly with the system dashboard.

Map Each Role's Journey

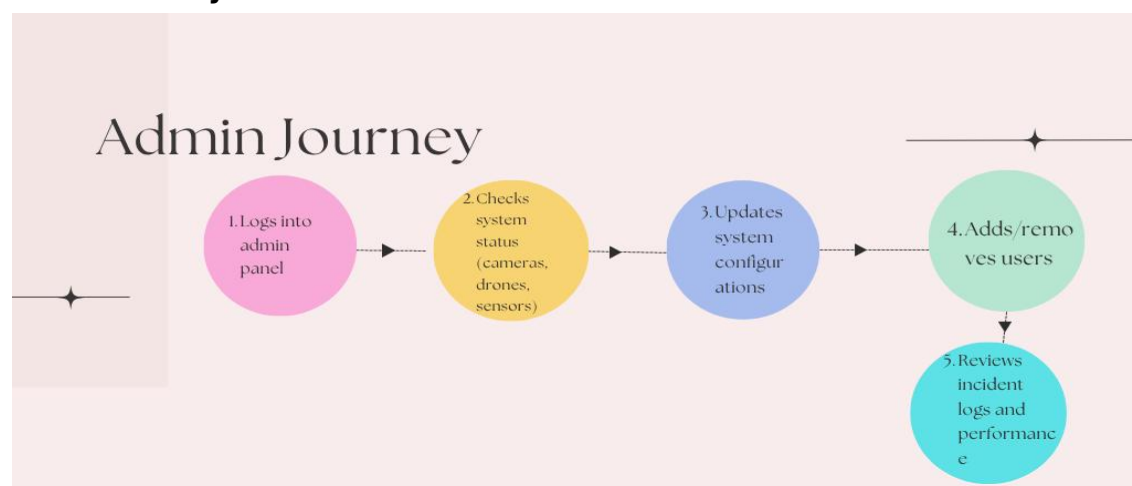
## Lifegaurd Journey



## Rescue team Journey



## Admin Journey



## Identify Key Input & Output Points

### User Inputs

- Login credentials (username & password)
- Button actions (e.g. “Deploy Drone”, “View Alert”)
- Search or filter options (e.g. select beach zone)
- Admin settings (device configuration, user roles)

### System Outputs

- Real-time video feeds from cameras/drones
- Alerts and notifications (distress detection, rip currents)
- GPS location of swimmer
- Predictive drift path visualization
- Confirmation messages (e.g. “Drone Deployed Successfully”)
- Status updates (active, resolved incident)

### Error & Edge Case Handling

- **Incorrect Login Details**  
→ System displays: “Invalid username or password”
- **No Internet / Connection Loss**  
→ System shows offline warning and disables live tracking
- **Drone Not Available / Battery Low**  
→ Alert: “Drone unavailable – select alternative”
- **No Swimmer Detected**  
→ System displays: “No active threats detected”
- **False Alert Detection**  
→ Lifeguard can dismiss alert and mark as false alarm
- **Sensor/Camera Failure**  
→ Admin receives maintenance notification
- **Overcrowded System / High Activity**  
→ System prioritizes high-risk alerts automatically

## Project Timeline

Project timeline starts after references!

## References

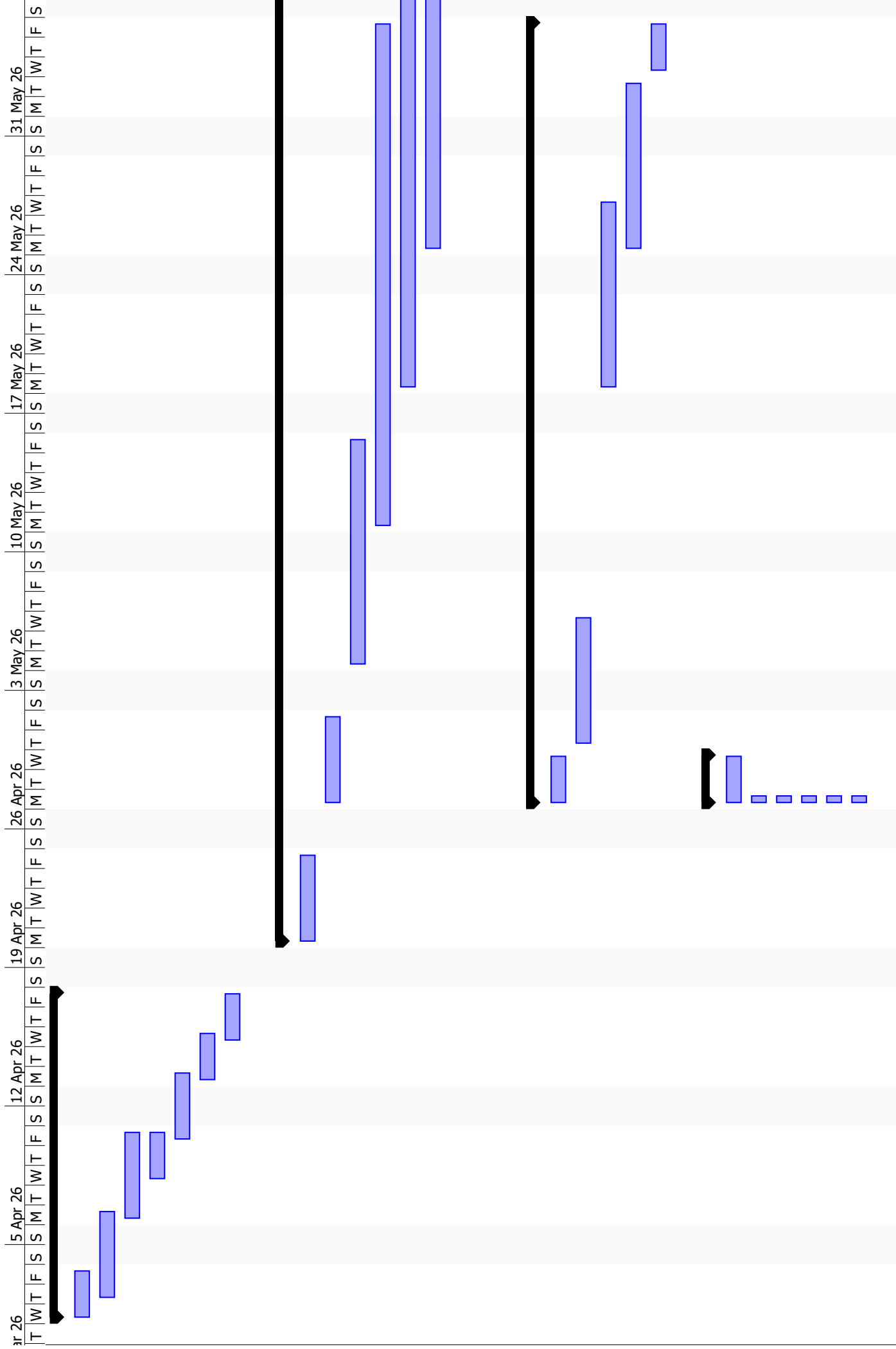
World Health Organization (WHO) (2021) *Global report on drowning: Preventing a leading killer*. Geneva: World Health Organization.

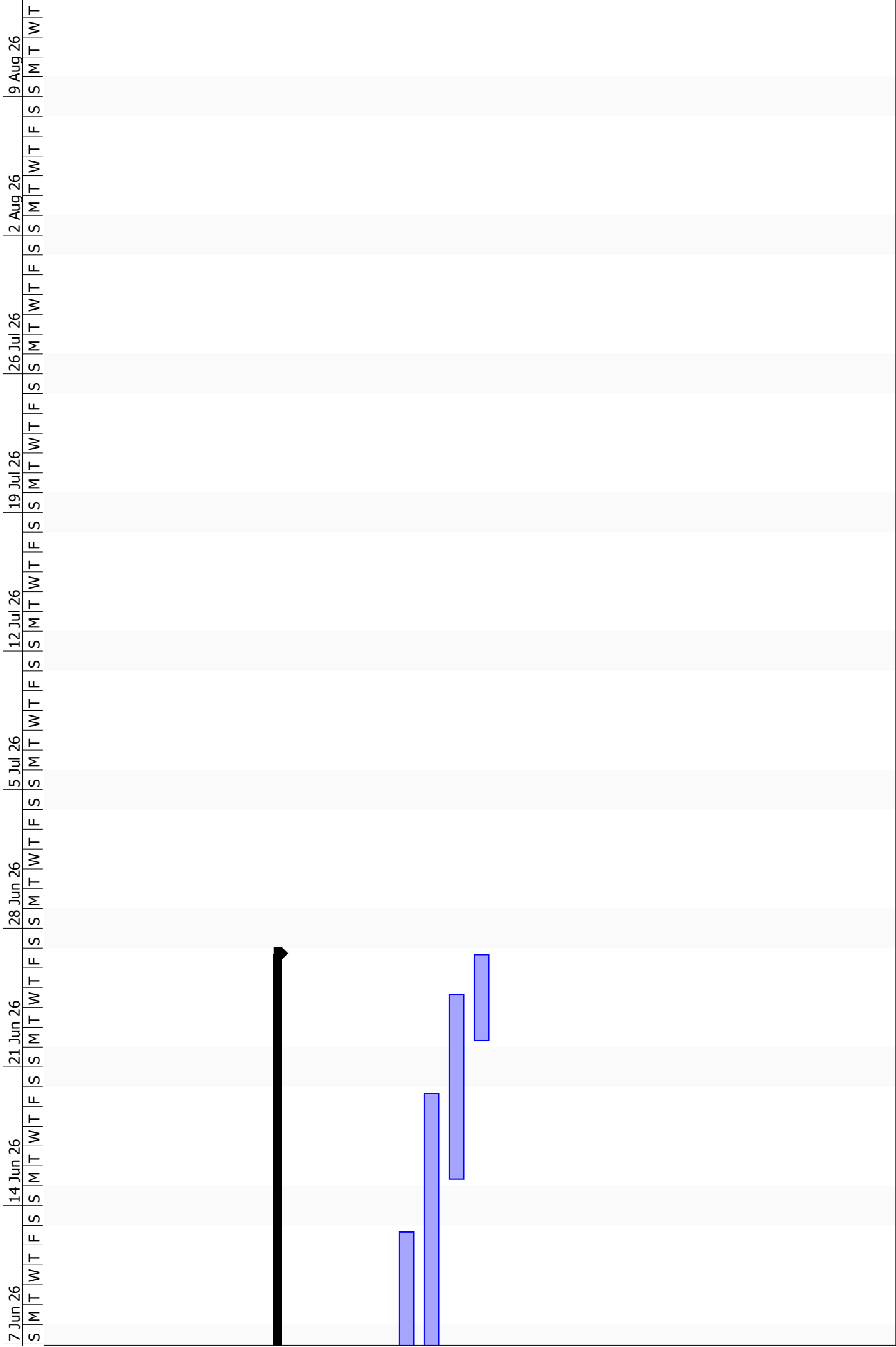
Aqua Wings (2023) *AI-powered drowning detection system*. Available at: <https://www.aquawings.com> (Accessed: 29 May 2026).

George Rescue Tech (2022) *Drone-assisted drowning prevention case study in George, South Africa*. Available at: <https://www.georgerescuetech.co.za> (Accessed: 29 May 2026).

Cygnus Project (2021) *IoT and AI/ML integrated drowning detection system*. Available at: <https://www.cygnusproject.org> (Accessed: 29 May 2026).

|    |   | Name                                      | Duration        | Start                    | Finish                   | Predecessors | Resource Names |  | 29 Mar 2025 |
|----|-------------------------------------------------------------------------------------|-------------------------------------------|-----------------|--------------------------|--------------------------|--------------|----------------|--|-------------|
|    |                                                                                     |                                           |                 |                          |                          |              |                |  |             |
| 1  |                                                                                     | <b>1. Project Planning &amp; Research</b> | <b>13 days?</b> | <b>2026/04/01, 08:00</b> | <b>2026/04/17, 17:00</b> |              |                |  |             |
| 2  |  | Define project scope                      | 3 days?         | 2026/04/01, 08:00        | 2026/04/03, 17:00        |              |                |  |             |
| 3  |  | Assign team responsibilities              | 3 days?         | 2026/04/02, 08:00        | 2026/04/06, 17:00        |              |                |  |             |
| 4  |  | Requirement analysis                      | 5 days?         | 2026/04/06, 08:00        | 2026/04/10, 17:00        |              |                |  |             |
| 5  |  | Risk analysis                             | 3 days?         | 2026/04/08, 08:00        | 2026/04/10, 17:00        |              |                |  |             |
| 6  |  | Select Agile methodology                  | 2 days?         | 2026/04/10, 08:00        | 2026/04/13, 17:00        |              |                |  |             |
| 7  |  | Set up GitHub                             | 3 days?         | 2026/04/13, 08:00        | 2026/04/15, 17:00        |              |                |  |             |
| 8  |  | Finalise project schedule                 | 3 days?         | 2026/04/15, 08:00        | 2026/04/17, 17:00        |              |                |  |             |
| 9  |  | <b>2. UI Development</b>                  | <b>50 days?</b> | <b>2026/04/20, 08:00</b> | <b>2026/06/26, 17:00</b> |              |                |  |             |
| 10 |  | Research UI requirements                  | 5 days?         | 2026/04/20, 08:00        | 2026/04/24, 17:00        |              |                |  |             |
| 11 |  | Create wireframes                         | 5 days?         | 2026/04/27, 08:00        | 2026/05/01, 17:00        |              |                |  |             |
| 12 |  | Design dashboard interface                | 10 days?        | 2026/05/04, 08:00        | 2026/05/15, 17:00        |              |                |  |             |
| 13 |  | Implement frontend layout                 | 20 days?        | 2026/05/11, 08:00        | 2026/06/05, 17:00        |              |                |  |             |
| 14 |  | Develop live drone feed display           | 20 days?        | 2026/05/18, 08:00        | 2026/06/12, 17:00        |              |                |  |             |
| 15 |  | Implement alert system visuals            | 20 days?        | 2026/05/25, 08:00        | 2026/06/19, 17:00        |              |                |  |             |
| 16 |  | Responsive UI testing                     | 8 days?         | 2026/06/15, 08:00        | 2026/06/24, 17:00        |              |                |  |             |
| 17 |  | UI bug fixing and refinement              | 5 days?         | 2026/06/22, 08:00        | 2026/06/26, 17:00        |              |                |  |             |
| 18 |  | <b>3. Drone Setup</b>                     | <b>30 days?</b> | <b>2026/04/27, 08:00</b> | <b>2026/06/05, 17:00</b> |              |                |  |             |
| 19 |  | Identify data requirements                | 3 days?         | 2026/04/27, 08:00        | 2026/04/29, 17:00        |              |                |  |             |
| 20 |  | Design database structure                 | 5 days?         | 2026/04/30, 08:00        | 2026/05/06, 17:00        |              |                |  |             |
| 21 |  | Implement real-time updates               | 8 days?         | 2026/05/18, 08:00        | 2026/05/27, 17:00        |              |                |  |             |
| 22 |  | Database connectivity testing             | 7 days?         | 2026/05/25, 08:00        | 2026/06/02, 17:00        |              |                |  |             |
| 23 |  | Database optimisation                     | 3 days?         | 2026/06/03, 08:00        | 2026/06/05, 17:00        |              |                |  |             |
| 24 |  | <b>4. Database Development</b>            | <b>3 days?</b>  | <b>2026/04/27, 08:00</b> | <b>2026/04/29, 17:00</b> |              |                |  |             |
| 25 |  | Identify data requirements                | 3 days?         | 2026/04/27, 08:00        | 2026/04/29, 17:00        |              |                |  |             |
| 26 |                                                                                     | Design database structure                 | 1 day?          | 2026/04/27, 08:00        | 2026/04/27, 17:00        |              |                |  |             |
| 27 |                                                                                     | Create database system                    | 1 day?          | 2026/04/27, 08:00        | 2026/04/27, 17:00        |              |                |  |             |
| 28 |                                                                                     | Implement real-time updates               | 1 day?          | 2026/04/27, 08:00        | 2026/04/27, 17:00        |              |                |  |             |
| 29 |                                                                                     | Database connectivity testing             | 1 day?          | 2026/04/27, 08:00        | 2026/04/27, 17:00        |              |                |  |             |
| 30 |                                                                                     | Database optimisation                     | 1 day?          | 2026/04/27, 08:00        | 2026/04/27, 17:00        |              |                |  |             |

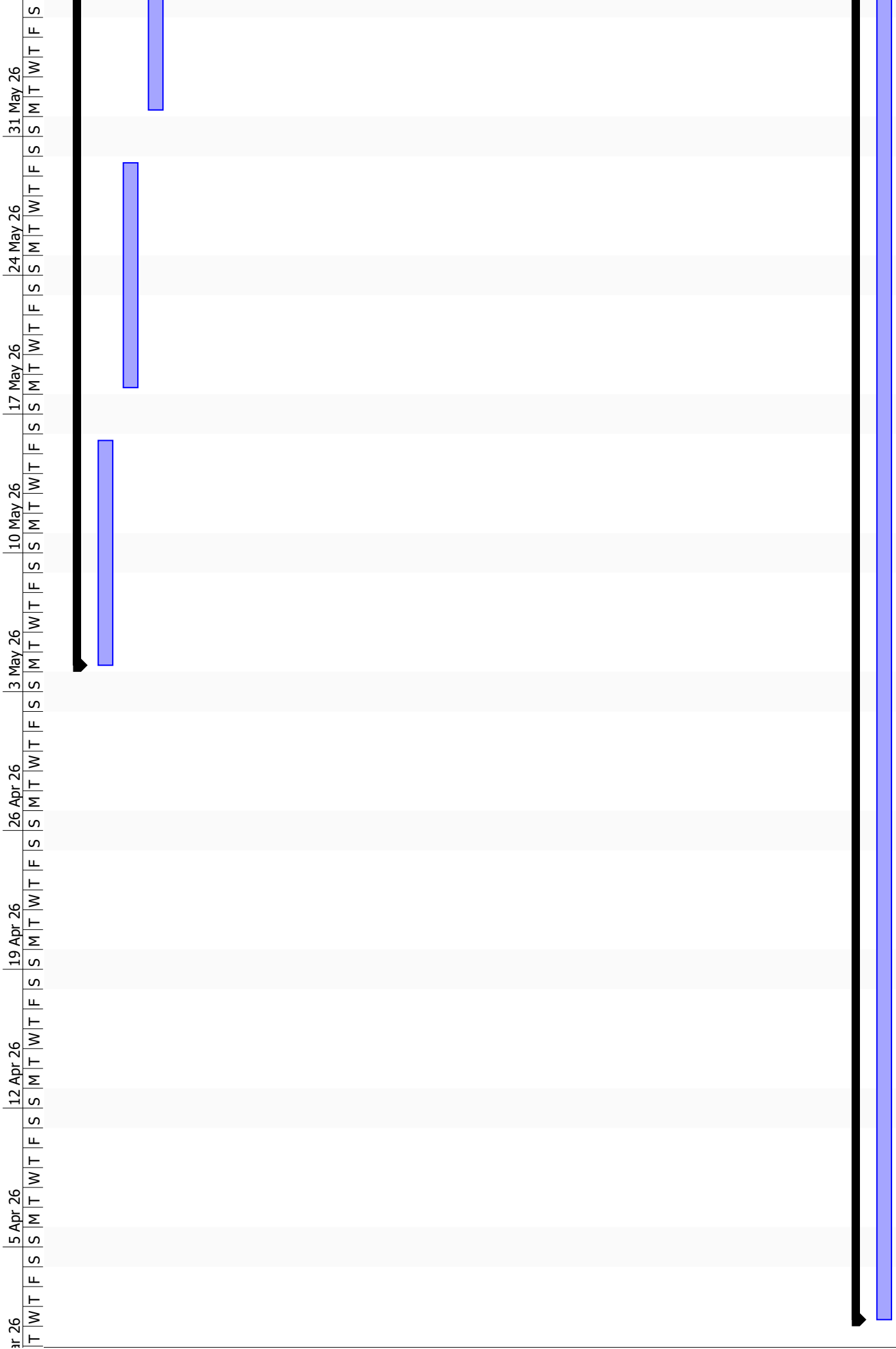


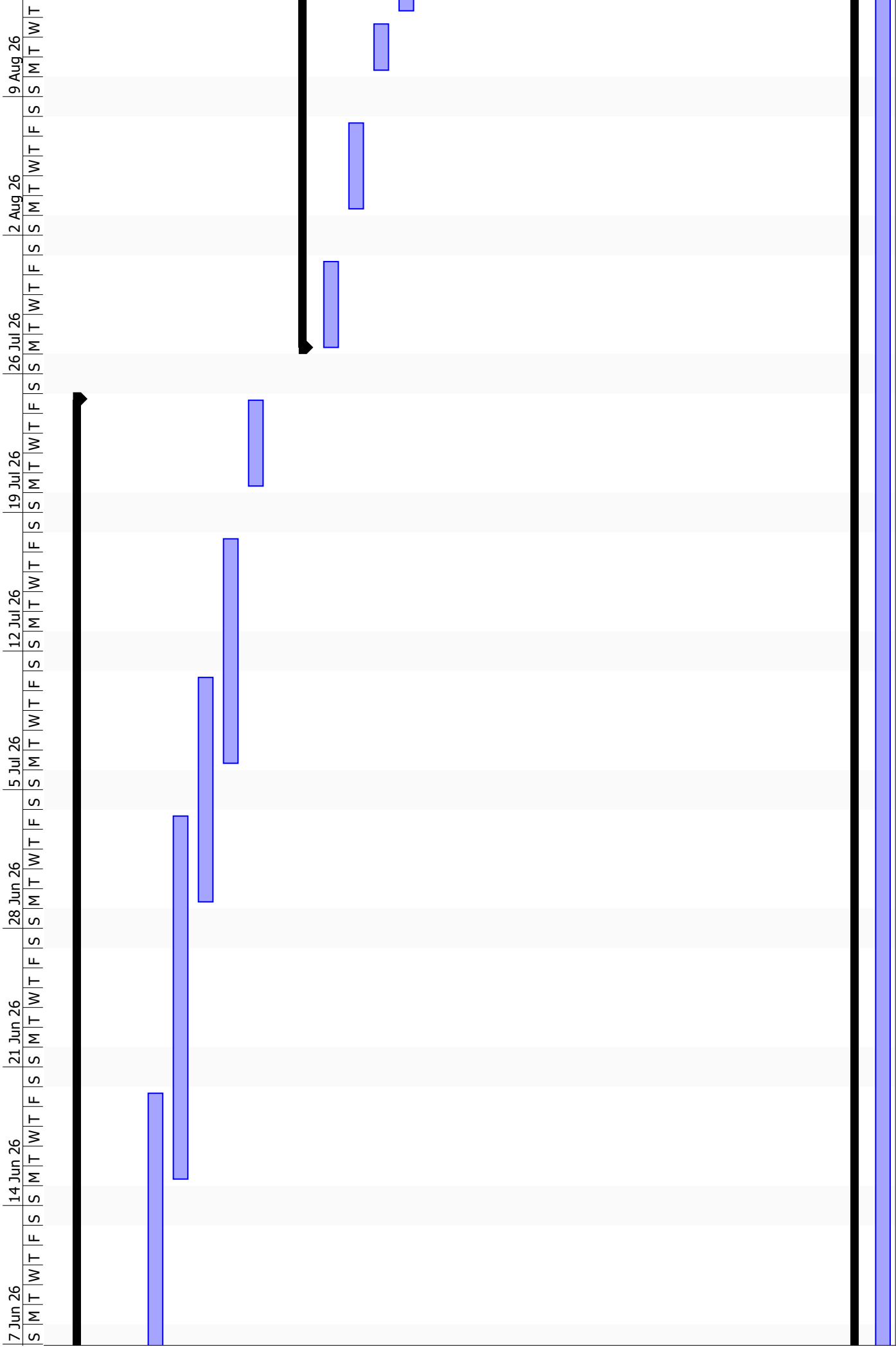


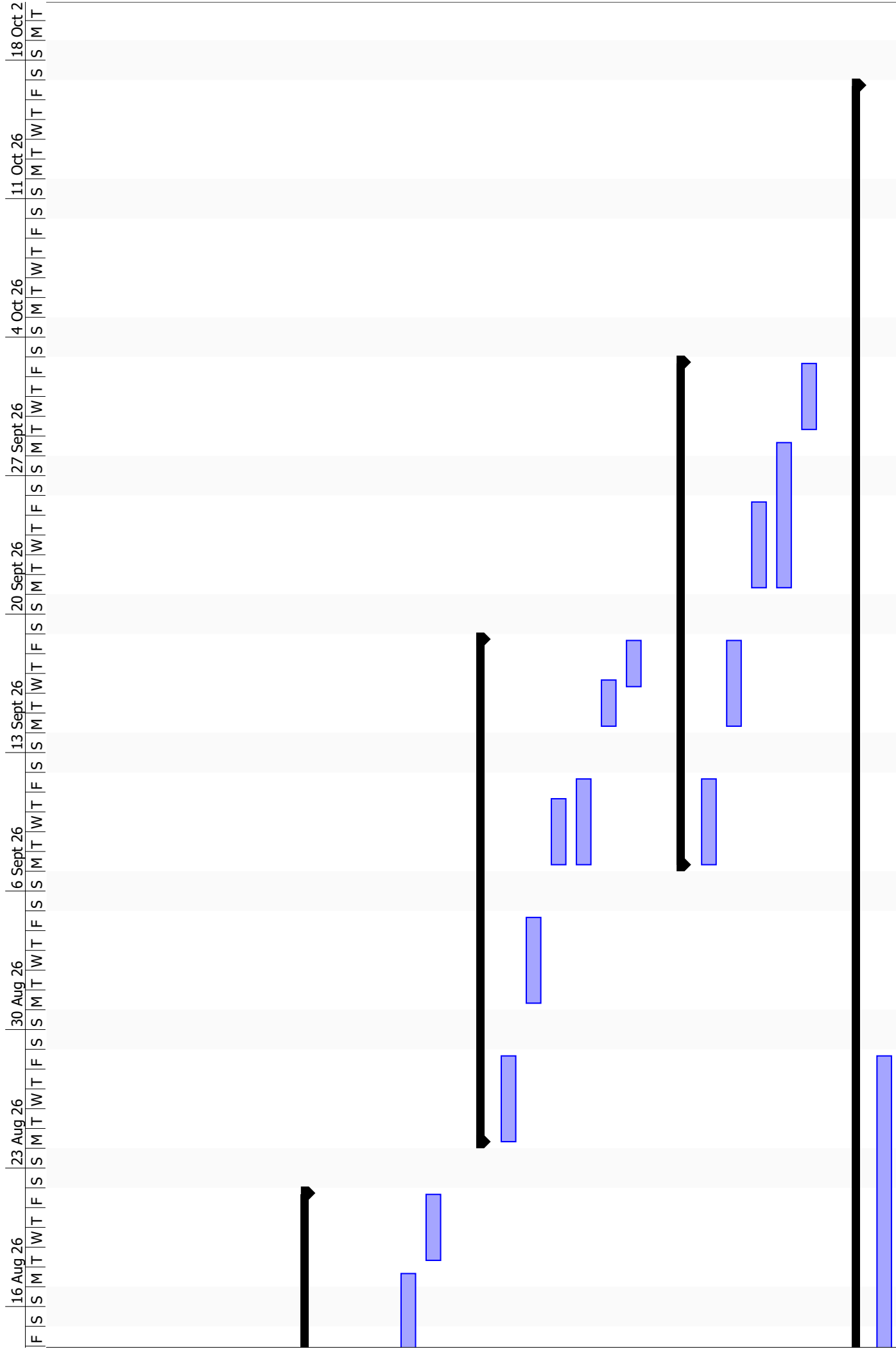




|    |     | Name                                         | Duration         | Start                    | Finish                   | Predecessors | Resource Names | 29 May<br>S   M |
|----|---------------------------------------------------------------------------------------|----------------------------------------------|------------------|--------------------------|--------------------------|--------------|----------------|-----------------|
| 31 |    | <b>☐5. Machine Learning Modelling</b>        | <b>60 days?</b>  | <b>2026/05/04, 08:00</b> | <b>2026/07/24, 17:00</b> |              |                |                 |
| 32 |    | Collect training datasets                    | 10 days?         | 2026/05/04, 08:00        | 2026/05/15, 17:00        |              |                |                 |
| 33 |    | Preprocess and clean data                    | 10 days?         | 2026/05/18, 08:00        | 2026/05/29, 17:00        |              |                |                 |
| 34 |    | Develop swimmer detection model              | 15 days?         | 2026/06/01, 08:00        | 2026/06/19, 17:00        |              |                |                 |
| 35 |    | Develop distress recognition model           | 15 days?         | 2026/06/15, 08:00        | 2026/07/03, 17:00        |              |                |                 |
| 36 |    | Develop drift prediction model               | 10 days?         | 2026/06/29, 08:00        | 2026/07/10, 17:00        |              |                |                 |
| 37 |    | Train and optimise models                    | 10 days?         | 2026/07/06, 08:00        | 2026/07/17, 17:00        |              |                |                 |
| 38 |    | Evaluate model accuracy                      | 5 days?          | 2026/07/20, 08:00        | 2026/07/24, 17:00        |              |                |                 |
|    |                                                                                       |                                              |                  |                          |                          |              |                |                 |
| 39 |    | <b>☐6. System Integration</b>                | <b>20 days?</b>  | <b>2026/07/27, 08:00</b> | <b>2026/08/21, 17:00</b> |              |                |                 |
| 40 |                                                                                       | Integrate ML with dashboard                  | 5 days?          | 2026/07/27, 08:00        | 2026/07/31, 17:00        |              |                |                 |
| 41 |    | Integrate drone systems                      | 5 days?          | 2026/08/03, 08:00        | 2026/08/07, 17:00        |              |                |                 |
| 42 |    | Connect GPS and tracking systems             | 3 days?          | 2026/08/10, 08:00        | 2026/08/12, 17:00        |              |                |                 |
| 43 |    | Integrate alert mechanisms                   | 3 days?          | 2026/08/13, 08:00        | 2026/08/17, 17:00        |              |                |                 |
| 44 |    | Resolve integration issues                   | 4 days?          | 2026/08/18, 08:00        | 2026/08/21, 17:00        |              |                |                 |
|    |                                                                                       |                                              |                  |                          |                          |              |                |                 |
| 45 |    | <b>☐7. Testing</b>                           | <b>20 days?</b>  | <b>2026/08/24, 08:00</b> | <b>2026/09/18, 17:00</b> |              |                |                 |
| 46 |                                                                                       | Unit testing                                 | 5 days?          | 2026/08/24, 08:00        | 2026/08/28, 17:00        |              |                |                 |
| 47 |    | Integration testing                          | 5 days?          | 2026/08/31, 08:00        | 2026/09/04, 17:00        |              |                |                 |
| 48 |    | ML accuracy testing                          | 4 days?          | 2026/09/07, 08:00        | 2026/09/10, 17:00        |              |                |                 |
| 49 |    | Drone communication testing                  | 5 days?          | 2026/09/07, 08:00        | 2026/09/11, 17:00        |              |                |                 |
| 50 |  | Simulated ocean-condition testing            | 3 days?          | 2026/09/14, 08:00        | 2026/09/16, 17:00        |              |                |                 |
| 51 |  | Bug fixing and retesting                     | 3 days?          | 2026/09/16, 08:00        | 2026/09/18, 17:00        |              |                |                 |
|    |                                                                                       |                                              |                  |                          |                          |              |                |                 |
| 52 |  | <b>☐8. Maintenance &amp; Optimisation</b>    | <b>20 days?</b>  | <b>2026/09/07, 08:00</b> | <b>2026/10/02, 17:00</b> |              |                |                 |
| 53 |                                                                                       | Refactor inefficient code                    | 5 days?          | 2026/09/07, 08:00        | 2026/09/11, 17:00        |              |                |                 |
| 54 |  | Improve UI usability                         | 5 days?          | 2026/09/14, 08:00        | 2026/09/18, 17:00        |              |                |                 |
| 55 |  | Optimise ML efficiency                       | 5 days?          | 2026/09/21, 08:00        | 2026/09/25, 17:00        |              |                |                 |
| 56 |  | Improve drone communication reli...          | 6 days?          | 2026/09/21, 08:00        | 2026/09/28, 17:00        |              |                |                 |
| 57 |  | Final optimisation review                    | 4 days?          | 2026/09/29, 08:00        | 2026/10/02, 17:00        |              |                |                 |
|    |                                                                                       |                                              |                  |                          |                          |              |                |                 |
| 58 |                                                                                       | <b>☐9. Documentation &amp; Presentati...</b> | <b>143 days?</b> | <b>2026/04/01, 08:00</b> | <b>2026/10/16, 17:00</b> |              |                |                 |
| 59 |                                                                                       | Prepare Agile report                         | 108 days?        | 2026/04/01, 08:00        | 2026/08/28, 17:00        |              |                |                 |





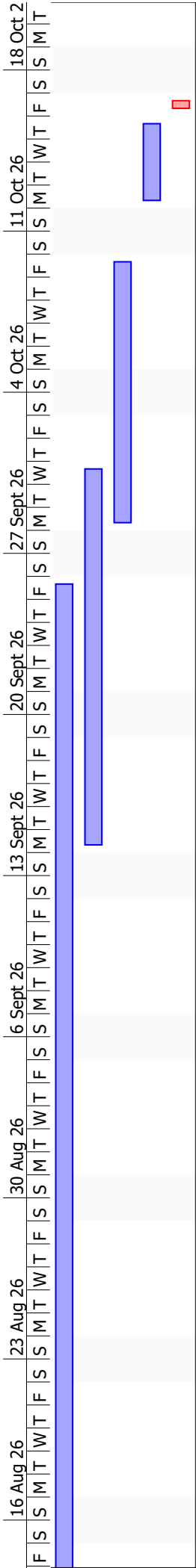




|    |                                                                                     | Name                           | Duration  | Start             | Finish            | Predecessors | Resource Names | 29 Mar<br>S   M |
|----|-------------------------------------------------------------------------------------|--------------------------------|-----------|-------------------|-------------------|--------------|----------------|-----------------|
| 60 |  | Technical documentation        | 115 days? | 2026/04/20, 08:00 | 2026/09/25, 17:00 |              |                |                 |
| 61 |  | Complete testing documentation | 13 days?  | 2026/09/14, 08:00 | 2026/09/30, 17:00 |              |                |                 |
| 62 |  | Prepare presentation slides    | 10 days?  | 2026/09/28, 08:00 | 2026/10/09, 17:00 |              |                |                 |
| 63 |  | Final presentation rehearsal   | 4 days?   | 2026/10/12, 08:00 | 2026/10/15, 17:00 |              |                |                 |
| 64 |  | Final submission               | 1 day?    | 2026/10/16, 08:00 | 2026/10/16, 17:00 |              |                |                 |







|   |           |   |   |   |   |   |   |          |   |   |   |   |   |   |          |   |   |   |   |   |   |    |   |   |   |   |   |   |
|---|-----------|---|---|---|---|---|---|----------|---|---|---|---|---|---|----------|---|---|---|---|---|---|----|---|---|---|---|---|---|
| 6 | 25 Oct 26 |   |   |   |   |   |   | 1 Nov 26 |   |   |   |   |   |   | 8 Nov 26 |   |   |   |   |   |   | 15 |   |   |   |   |   |   |
|   | W         | T | F | S | S | M | T | W        | T | F | S | S | M | T | W        | T | F | S | S | M | T | W  | T | F | S | S | M | T |
|   |           |   |   |   |   |   |   |          |   |   |   |   |   |   |          |   |   |   |   |   |   |    |   |   |   |   |   |   |